

Facultad de Ciencias Exactas, Ingeniería y Agrimensura



FCEIA

TUIA NLP 2025

Trabajo Final: RAG + Agente ReAct

Alumno: Alejandro Armas

Docentes:

- Alan Geary
 - Juan Pablo Manson
 - Constantino Ferrucci
-

Introducción

El presente trabajo tiene como finalidad el desarrollo de un sistema conversacional para la empresa de electrodomésticos ficticia "ElectroHogar". El proyecto se divide en dos objetivos técnicos principales:

1. Desarrollar un sistema de Retrieval-Augmented Generation (RAG) que mediante un pipeline de retrieval, usando un clasificador de intención, pueda responder preguntas pertinentes, tomando fuentes de datos tabulares, vectoriales y de grafos.
2. Desarrollar un agente autónomo bajo el paradigma ReAct, capaz de combinar y orquestar las herramientas desarrolladas y usar las fuentes de datos mencionadas, generando mejores respuestas y también sumando la habilidad de generar gráficos.

Ejemplos de preguntas a responder:

- Información sobre usos de productos: "¿Cómo uso mi licuadora para hacer smoothies?"
- Información sobre precios de productos: "¿Cuáles son las licuadoras de menos de \$200?"
- Información sobre reseñas de usuarios: "¿Qué opinan los usuarios de esta cafetera?"
- Recomendación sobre productos: "Quiero una licuadora con buenas reseñas"
- Respuestas a preguntas frecuentes: "¿Qué voltaje requiere el rallador digital eléctrico?"

- Respuestas sobre productos: "¿Qué productos están relacionados con la categoría Cocina?"

Fuentes de información

Para abordar la complejidad y variedad de las consultas, fue necesario analizar y seleccionar adecuadamente la tecnología óptima para cada fuente de información.

Distribución de las fuentes de información entre los tipos de fuentes de datos.

Tipo de Base de Datos	Fuentes utilizadas
Vectorial (RAG)	manuales_productos, reseñas_usuarios, faqs.json
Tabular (Pandas)	ventas_historicas.csv, productos.csv, tickets_soporte.csv, inventario_sucursales.csv
Grafo (Neo4j)	ventas_historicas.csv, productos.csv, tickets_soporte.csv, vendedores.csv, devoluciones.csv

Los manuales de los productos son archivos `.md` que mantienen una estructura común, como indicar *Marca* y *Modelo* dentro del primer y único `#header` de nivel 1. Extraigo dicha información con expresiones regulares para poder inyectarla como contexto y metadata (mejorando los embeddings). También usé un splitter de markdown, separando en cada *sección* y *subsección* (headers de nivel 2 y 3) de cada manual. Luego proceso esos splits con otro splitter para limitar los chunks a 500 caracteres.

Las reseñas de los productos son archivos `.txt` que también mantienen una estructura común, donde primero indican un set de metadatos y luego el texto crudo de la reseña. De nuevo extraigo los datos de utilidad mediante expresiones regulares para poder utilizarlos como contexto y metadata.

El archivo `faqs.json` es fácilmente procesable, teniendo explicitada la respuesta a cada pregunta y la metadata del producto. También lo proceso adecuadamente para generar buenos embeddings.

Los demás son archivos `.csv` que distribuyo entre la base de datos tabular y la de grafos, según la naturaleza de la información.

`ventas_historicas.csv`, `productos.csv` y `tickets_soporte.csv` los incluyo tanto en la base de grafos como la tabular, para poder realizar consultas como **Licadoras de menos de \$300** (tabular) y también **Usuarios que compraron la licuadora X y luego abrieron un ticket de reclamo** (grafos)

Para la base de grafos incluyo también `devoluciones.csv` y `vendedores.csv`, permitiendo modelar que un *nodo usuario devolvió* un *nodo producto*, o que un *nodo vendedor vendió* un *nodo producto*.

Para la base tabular también incluyo `inventario_sucursales.csv` que contiene información estructurada cuantitativa, perfecta para modelar tabularmente. Permitiendo responder consultas como **¿Qué sucursal tiene menos stock del producto X?**

Procesamiento de las fuentes de información

Para trabajar ordenadamente, implementé clases específicas para cada tipo de fuente de datos, las cuales decidí llamar *Gestores*. Cada gestor se encarga de la carga, preprocesamiento y consultas a su respectiva fuente de datos, facilitando el uso de las mismas tanto en el RAG como en el Agente ReAct.

Gestor Vectorial

Se encarga de cargar las fuentes de datos vectoriales, procesarlas y generar los embeddings. También implementa la funcionalidad de búsqueda vectorial pura (para el RAG) e híbrida (vectorial + palabras clave) con rerank (para el Agente ReAct).

Utiliza Chroma como base de datos vectorial local, y el modelo **gemma-300m** para generar los embeddings.

La búsqueda híbrida se implementa combinando la búsqueda vectorial de chroma con búsqueda por palabras clave mediante **bm25** y luego usando un reranker basado en el modelo **mmarco-mMiniLMv2-L12-H384**, que filtra los resultados para mejorar la relevancia de los fragmentos recuperados.

Problemas encontrados

- Al utilizar Chroma en Google Colab, noté que al reiniciar el entorno de ejecución, se perdían los datos almacenados en memoria. Para solucionar este problema, implementé una funcionalidad de persistencia que guarda y carga la base de datos vectorial desde el 'disco' (google drive).

Gestor Tabular

Se encarga de cargar las fuentes de datos tabulares, procesarlas y generar un schema dinámico de los datos (para el LLM). También implementa la funcionalidad de búsqueda tabular mediante consultas generadas por el LLM y la funcionalidad de generación de gráficos a partir de los datos tabulares mediante código generado por el LLM (para el Agente ReAct).

Utiliza Pandas para manejar los datos tabulares y matplotlib para generar los gráficos. Los prompts para generar las consultas tabulares y el código matplotlib para generar gráficos se diseñaron cuidadosamente para asegurar que el LLM genere código correcto y eficiente.

Gestor de Grafos

Se encarga de cargar las fuentes de datos de grafos, procesarlas y generar el grafo con los nodos y relaciones correspondientes, además de generar un esquema dinámico de los datos (para el LLM). También implementa la funcionalidad de búsqueda de grafos mediante consultas cypher generadas por el LLM.

Utiliza Neo4j para manejar los datos de grafos ya que permite consultas cypher y además al ser una base remota, pude evitar regenerar el grafo en cada ejecución del entorno colab, agilizando el desarrollo.

Problemas encontrados

- Inicialmente, el esquema del grafo era estático (hardcodeado), y si bien los resultados no eran malos, detecté que algunas consultas no eran respondidas correctamente. Por ejemplo: **"Quién es el cliente con mayor cantidad de tickets de máxima severidad?"**, no me respondía con ningún cliente, ya que no "entendía" qué valor de severidad significaba "máxima" (Los valores posibles de severidad son "baja", "media" y "alta"). Esto se debía a que el esquema estático no incluía esa información, y el LLM no podía inferirla correctamente. La solución fue generar el esquema dinámicamente con el grafo ya

generado, utilizando una técnica llamada *Introspection*, donde se consulta al grafo por su estructura, y si existen propiedades con cardinalidad limitada (como en este caso la propiedad "severidad" del nodo "ticket"), se incluyen esos valores posibles en el esquema generado. De esta forma el LLM que genera las consultas cypher tiene toda la información necesaria para generar consultas correctas. (función *refrescar_schema* del Gestor de Grafos). Ejemplo: screenshot mostrando el correcto funcionamiento gracias a la introspección, puede identificar que "alta" es el valor máximo de severidad.

```
query = "¿Quién es el usuario con mayor cantidad de tickets de máxima severidad?"
consultar_agente(query, steps=True)

Paso 1
Pensamiento: Action: graph_search
Action Input: Usuario con mayor cantidad de tickets de máxima severidad
Herramienta: graph_search
Input: Usuario con mayor cantidad de tickets de máxima severidad
Resultado: [{"Usuario": "Laura Hernández", "CantidadTicketsAltaSeveridad": 4}, {"Usuario": "Miranda Castro", "CantidadTicketsAltaSeveridad": 4}, {"Usuario": "Laura Peralta", "CantidadTicketsAltaSeveridad": 1}]
-----
Los usuarios con mayor cantidad de tickets de máxima severidad son Laura Hernández y Miranda Castro, ambos con 4 tickets.
```

Retrieval-Augmented Generation (RAG)

El sistema RAG implementa un pipeline de retrieval que utiliza un clasificador de intención para determinar qué gestor utilizar para responder la consulta del usuario, y luego utiliza un LLM para generar la respuesta final basada en los fragmentos recuperados por el gestor correspondiente.

Clasificador de intención

SVM + TF-IDF (scikit-learn, local)

Entrené un clasificador de intención simple usando tf-idf y svm con un conjunto de datos sintético pequeño (10 ejemplos por clase) para clasificar las consultas del usuario en tres categorías: *vectorial*, *tabular* y *grafo*. Obteniendo un desempeño regular en las métricas de evaluación. Particularmente predice muy mal la clase *tabular*.

```
print(classification_report(y_test, y_pred_custom))
```

	precision	recall	f1-score	support
grafo	0.67	0.50	0.57	4
tabular	0.25	0.33	0.29	3
vectorial	0.50	0.50	0.50	2
accuracy			0.44	9
macro avg	0.47	0.44	0.45	9
weighted avg	0.49	0.44	0.46	9

LLM (Gemini 2.5 Flash, cloud)

Utilizando few-shot, incluyendo ejemplos en el prompt, sobre los mismos datos de testeo, el llm logró un desempeño perfecto en las métricas de evaluación, superando ampliamente al clasificador svm.

```
print(classification_report(y_test, y_pred_llm))
```

	precision	recall	f1-score	support
grafo	1.00	1.00	1.00	4
tabular	1.00	1.00	1.00	3
vectorial	1.00	1.00	1.00	2
accuracy			1.00	9
macro avg	1.00	1.00	1.00	9
weighted avg	1.00	1.00	1.00	9

Prompt utilizado (few-shot)

```
"""
Eres un clasificador de intención para una empresa de electrodomesticos
Tu trabajo es categorizar la pregunta del usuario en UNA de estas 3
categorías:

1. VECTORIAL: Para preguntas sobre CÓMO usar algo, manuales, garantías,
opiniones, reseñas (texto cualitativo).
2. TABULAR: Para preguntas sobre PRECIOS, stock, cantidades, cálculos,
estadísticas de ventas (datos numéricos exactos).
3. GRAFO: Para preguntas sobre RELACIONES, "quién compró qué", tickets de
soporte, recomendaciones, usuarios específicos.

EJEMPLOS:
- "Cómo limpio el filtro?" -> VECTORIAL
- "Precio de la licuadora" -> TABULAR
- "Usuarios que compraron esto" -> GRAFO
- "Quejas sobre el motor" -> VECTORIAL
- "Ventas totales en marzo" -> TABULAR
- "Clientes con tickets abiertos" -> GRAFO

PREGUNTA: {pregunta}

Responde con nada más que la palabra de la categoría (VECTORIAL, TABULAR o
GRAFO).
"""
```

Veredicto

Debido al excelente desempeño del LLM, decidí utilizarlo como clasificador de intención en el sistema RAG, pero implementandolo de forma que en caso de que el LLM falle por cualquier motivo, se utilice el clasificador local svm como respaldo.

Pipeline RAG

El pipeline del RAG funciona de la siguiente manera:

1. El usuario realiza una consulta.
2. El clasificador de intención determina qué gestor utilizar (vectorial, tabular o grafo).
3. Se pasa la consulta al gestor correspondiente, que recupera los fragmentos relevantes de las fuente de datos.
4. El LLM genera la respuesta final basada en los fragmentos recuperados, el historial de la conversación y la pregunta del usuario.

Implementación

La implementación del RAG es simple, acatando el orden de pasos definidos en el Pipeline y utilizando langchain.

El historial de la conversación se implementó de forma simple, utilizando una lista de tuplas (pregunta, respuesta) e inyectando los ultimos 3 elementos de dicha lista (últimas 3 interacciones) en el prompt del LLM.

Prompt utilizado LLM

El prompt utilizado para generar la respuesta final. Incluyendo la información recuperada, el historial de la conversación y la pregunta del usuario. Aclara las reglas a seguir para obtener una respuesta coherente y prevenir respuestas inventadas, exigiendo más información en caso de necesitarla.

```
"""
Eres el asistente de IA de ElectroHogar, una casa de electrodomésticos.

HISTORIAL RECIENTE:
{historial}

PREGUNTA DEL USUARIO:
{pregunta}

INFORMACIÓN RECUPERADA ({fuente}):
{contexto}

REGLAS:
- INFORMACIÓN RECUPERADA es el resultado exacto de una consulta a alguna db (pandas, chroma o neo4j).
- Usa el historial reciente para comprender el contexto de la pregunta si esta no aclara modelo ni nombre.
- Si necesitas más informacion, rehústate a responder y pide mejorar la pregunta.
- Si la información recuperada no es útil o está vacía, dilo claramente.
- Si la información es una tabla o lista, adáptala a texto natural.
"""
```

Resultados de ejecución

```
print(rag.responder("¿Cómo uso mi licuadora para hacer smoothies?"))
```

¡Hola! Para usar tu licuadora Compacto de ChefMaster (modelo P0004) para preparar un smoothie de frutas, sigue estos sencillos pasos:

1. ****Prepara las frutas:**** Lava y corta las frutas en trozos medianos (aproximadamente de 2 a 3 cm).
2. ****Agrega los líquidos:**** Coloca primero los ingredientes líquidos en la jarra de la licuadora (como leche, yogurt o jugo).
3. ****Añade las frutas y el hielo:**** Luego, agrega las frutas cortadas y el hielo.
4. ****Cierra la tapa:**** Asegúrate de que la tapa esté bien cerrada y sellada herméticamente.
5. ****Inicia el licuado:**** Comienza a licuar a velocidad baja (nivel 1-2) durante unos 10 segundos.
6. ****Aumenta la velocidad:**** Gradualmente, aumenta la velocidad a un nivel alto (nivel 4-5) hasta obtener la consistencia deseada.

¡Así de fácil tendrás tu smoothie listo!

```
print(rag.responder("¿Cuáles son las licuadoras de menos de $200?"))
```

Las licuadoras con un precio menor a \$200 son:

- * Licuadora Basic (TechHome) - \$150.00
- * Compacto Licuadora (HomeChef) - \$180.00
- * Licuadora Viajera (TechHome) - \$120.00

```
print(rag.responder("Me compré la cafetera P0121, que voltaje necesita?"))
```

La cafetera P0121 necesita un voltaje de 12V.

```
print(rag.responder("¿Qué opinan los usuarios de esta cafetera?"))
```

Lo siento, no tengo opiniones de usuarios para la cafetera P0121. La información que he encontrado corresponde al modelo P0122.

Esta respuesta demostró el buen desempeño del RAG, por un lado entendió que le estaba haciendo otra pregunta sobre la P0121 por historial, y por el otro me respondió honestamente que no tiene reseñas de usuarios sobre dicho modelo.

```
print(rag.responder("Quiero una licuadora con buenas reseñas"))
```

Lo siento, no he encontrado licuadoras con reseñas que se consideren "buenas" en la información disponible. Las reseñas que encontré para los modelos P0001, P0005 y P0006 indican que tienen "sus pros y contras" y que "por el precio está bien, pero no esperen maravillas".

```
print(rag.responder("¿Qué voltaje requiere el rallador digital eléctrico?"))
```

El rallador digital eléctrico requiere un voltaje de 110V.

En este caso habría sido mejor que nos pregunte que modelo de rallador específico tenemos.

```
print(rag.responder("Muestrame 10 productos de la categoría Cocina"))
```

¡Claro! Aquí tienes 10 productos de la categoría Cocina que tenemos disponibles:

- * Licuadora
- * Plus Licuadora Pro
- * Compacto Licuadora
- * Ultra Licuadora
- * Procesadora
- * Horno Eléctrico
- * Batidora de Mano
- * Tostadora
- * Cafetera
- * Microondas


```
¿Hay algún electrodoméstico de cocina en particular que te interese, o tienes alguna otra consulta?
```

```
print(rag.responder("¿Qué usuarios devolvieron todos los productos que compraron?"))
```

```
¡Hola! Los usuarios que devolvieron todos los productos que compraron son: Lucas Ramírez, Gabriel Benítez, Julieta Silva, Agustina Flores y Rodrigo Rivera.
```

Comentarios

El sistema RAG demostró ser capaz de manejar una variedad de consultas relacionadas con los productos de ElectroHogar correctamente, utilizando las fuentes de datos y el historial de la conversación de manera efectiva para proporcionar respuestas precisas y relevantes.

Agent ReAct

El agente ReAct implementa un sistema autónomo que combina y orquesta las herramientas desarrolladas (los gestores) para responder consultas más complejas del usuario, incluyendo la generación de gráficos a partir de datos tabulares.

Implementación

La implementación fue simple gracias a la librería **langchain**, que facilita la creación de agentes ReAct utilizando LLMs y herramientas personalizadas y al hecho de que ya tenía los gestores implementados. Cada tool del agente es un wrapper de un método de cada gestor, que recibe la query del usuario y devuelve la respuesta del gestor correspondiente.

Una vez definidas las tools, se crea el agente ReAct utilizando el LLM como "cerebro", que interpreta la query del usuario, decide qué tools utilizar y en qué orden, y genera la respuesta final.

La implementación del historial, al igual es que en el RAG, se realizó mediante una lista de tuplas que luego se pasa dentro del prompt.

Prompt ReAct utilizado

Se utilizó el siguiente prompt, que le indica al agente como debe ser el proceso de pensamiento y acción, incluyendo las herramientas disponibles (tools) y el historial de la conversación.

Además de una serie de reglas mandatorias para mejorar la respuesta generada y el formato de la misma. También garantizar que responda en el mismo idioma en el que se le consultó y que tenga en cuenta el

historial para identificar un posible sujeto tácito.

```
"""
```

Responde la siguiente pregunta teniendo en cuenta el historial de la conversación. Tienes acceso a las siguientes herramientas:

```
{tools}
```

Respetá el siguiente formato:

Question: la pregunta que debes responder

Thought: piensa en tu próximo paso, justifica la elección de tu herramienta

Action: la acción a ejecutar, debe ser una de [{tool_names}]

Action Input: string conteniendo la entrada para la acción en lenguaje natural

Observation: el resultado de la acción

... (El flujo Thought/Action/Action Input/Observation puede repetirse N veces)

Thought: ahora sé la respuesta final

Final Answer: la respuesta final a la pregunta

REGLAS MANDATORIAS:

- SIEMPRE mira el historial para completar la pregunta.
- SIEMPRE usa al menos una de las herramientas disponibles.
- Agrega el símbolo \$ si vas a dar un precio.
- Responde siempre en el mismo idioma que la pregunta.
- Si no obtuviste resultados al usar una herramienta, intenta de nuevo con otros límites o intenta con otra herramienta.
- Si la pregunta no está relacionada con electrodomésticos debes rehusarte a responder.
- Si necesitas mas información para responder correctamente la pregunta, pídelo claramente.

```
History: {historial}
```

```
Question: {input}
```

```
Thought: {agent_scratchpad}
```

```
"""
```

Función de consulta

Función implementada para consultar al agente ReAct, agregando el historial reciente al prompt y guardando la nueva interacción. Admite el flag `steps=True` para mostrar los pasos intermedios del proceso de razonamiento del agente.

```
def consultar_agente(query, steps=False):
    # Formateo simple del historial con últimas 3 interacciones.
    historial_reciente = "\n".join([f"User: {x[0]} | Bot: {x[1]}" for x in
    historial_agente[-3:]])

    rta = agent_executor.invoke({
```

```

        "input": query,
        "historial": historial_reciente
    })
    historial_agente.append((query, rta['output']))

    if steps:
        for i, (action, observation) in enumerate(rta["intermediate_steps"],
start=1):
            print(f"\n\033[93mPaso {i}\033[0m")
            print(f"  \nPensamiento: {action.log}")
            print(f"  \nHerramienta: {action.tool}")
            print(f"  \nInput: {action.tool_input}")
            print(f"  \nResultado: {observation}")

            print('\n' + '-'*20 + '\n')

    print(rta['output'])

```

Resultados de ejecución

```
# Prueba de multiples consultas en un mismo prompt, activo steps para ver
el razonamiento completo
```

```

query = "¿Cuál es el precio del producto modelo P0005 y qué dicen las
reseñas sobre él?, además dame el nombre de los usuarios que la compraron y
la devolvieron"
consultar_agente(query, steps=True)

```

Paso 1

```

Pensamiento: Question: ¿Cuál es el precio del producto modelo P0005 y qué
dicen las reseñas sobre él?, además dame el nombre de los usuarios que la
compraron y la devolvieron
Thought: El usuario quiere saber el precio, las reseñas, y los usuarios que
compraron y devolvieron el producto P0005.
Primero, usaré `table_search` para obtener el precio del producto P0005.
Action: table_search
Action Input: precio del producto P0005

```

```
Herramienta: table_search
```

```
Input: precio del producto P0005
```

```
Resultado: 2602.78
```

Paso 2

```
Pensamiento: Ahora tengo el precio del producto P0005. El siguiente paso es
```

obtener las reseñas y los usuarios que compraron y devolvieron el producto.
Para las reseñas, usaré `doc\_search`.

Action: doc_search

Action Input: reseñas del producto P0005

Herramienta: doc_search

Input: reseñas del producto P0005

Resultado: Fuente: reseña - Licuadora

Reseña de Producto: Licuadora Modelo: P0005

OPINIÓN:

Buenas! Tiene sus pros y contras con Licuadora. Tiene cosas buenas y cosas malas. Por el precio está bien, pero no esperen maravillas. Espero les sirva!

Fuente: reseña - Licuadora

Reseña de Producto: Licuadora Modelo: P0005

OPINIÓN:

Hola! Normal, sin sorpresas con Licuadora. Tiene cosas buenas y cosas malas. Por el precio está bien, pero no esperen maravillas. Gracias!

Fuente: reseña - Licuadora

Reseña de Producto: Licuadora Modelo: P0005

OPINIÓN:

Les cuento... Muy buena calidad, estoy feliz con Licuadora. Lo compré hace dos meses y es muy preciso, además de duradero. Ya se lo recomendé a varios amigos. Cualquier duda avisen.

Paso 3

Pensamiento: Action: graph_search

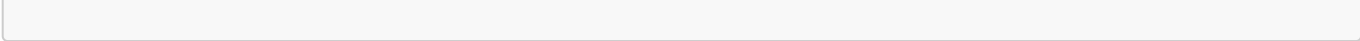
Action Input: usuarios que compraron y devolvieron el producto P0005

Herramienta: graph_search

Input: usuarios que compraron y devolvieron el producto P0005

Resultado: [{ 'u.nombre': 'Sofía Cruz' }, { 'u.nombre': 'Emma Torres' },
{ 'u.nombre': 'Lucas Benítez' }]

El precio del producto modelo P0005 es \$2602.78. Las reseñas sobre este producto son variadas: algunos usuarios lo consideran de calidad normal, sin sorpresas y con pros y contras por el precio, mientras que otro usuario está muy feliz con la calidad, lo describe como preciso y duradero, y lo recomienda. Los usuarios que compraron y devolvieron el producto P0005 son Sofía Cruz, Emma Torres y Lucas Benítez.



```
# Prueba de contexto e historial

consultar_agente("Que dice la peor reseña?", steps=True)
```

Paso 1

Pensamiento: Thought: El usuario quiere saber el contenido de la "peor reseña" para el producto P0005. En el historial ya se mencionaron reseñas variadas, incluyendo algunas no tan positivas. Necesito usar `doc\_search` para intentar obtener el texto específico de la reseña más negativa o que se considere la peor, basándome en el modelo P0005.

Action: doc_search

Action Input: ¿Cuál es la peor reseña del producto modelo P0005?

Herramienta: doc_search

Input: ¿Cuál es la peor reseña del producto modelo P0005?

Resultado: Fuente: reseña - Licuadora
Reseña de Producto: Licuadora Modelo: P0005
OPINIÓN:
Buenas! Tiene sus pros y contras con Licuadora. Tiene cosas buenas y cosas malas. Por el precio está bien, pero no esperen maravillas. Espero les sirva!

Fuente: reseña - Licuadora
Reseña de Producto: Licuadora Modelo: P0005
OPINIÓN:
Hola! Lamentablemente debo decir que la calidad es muy mala con Licuadora. Es muy lento y difícil de limpiar. Se rompió a los pocos días de uso. Espero les sirva!

Fuente: reseña - Licuadora
Reseña de Producto: Licuadora Modelo: P0005
OPINIÓN:
Buenas! Lamentablemente debo decir que decepcionante, no vale lo que cuesta con Licuadora. Es muy consume mucho y caro. No vale lo que cuesta. Cualquier duda avisen.

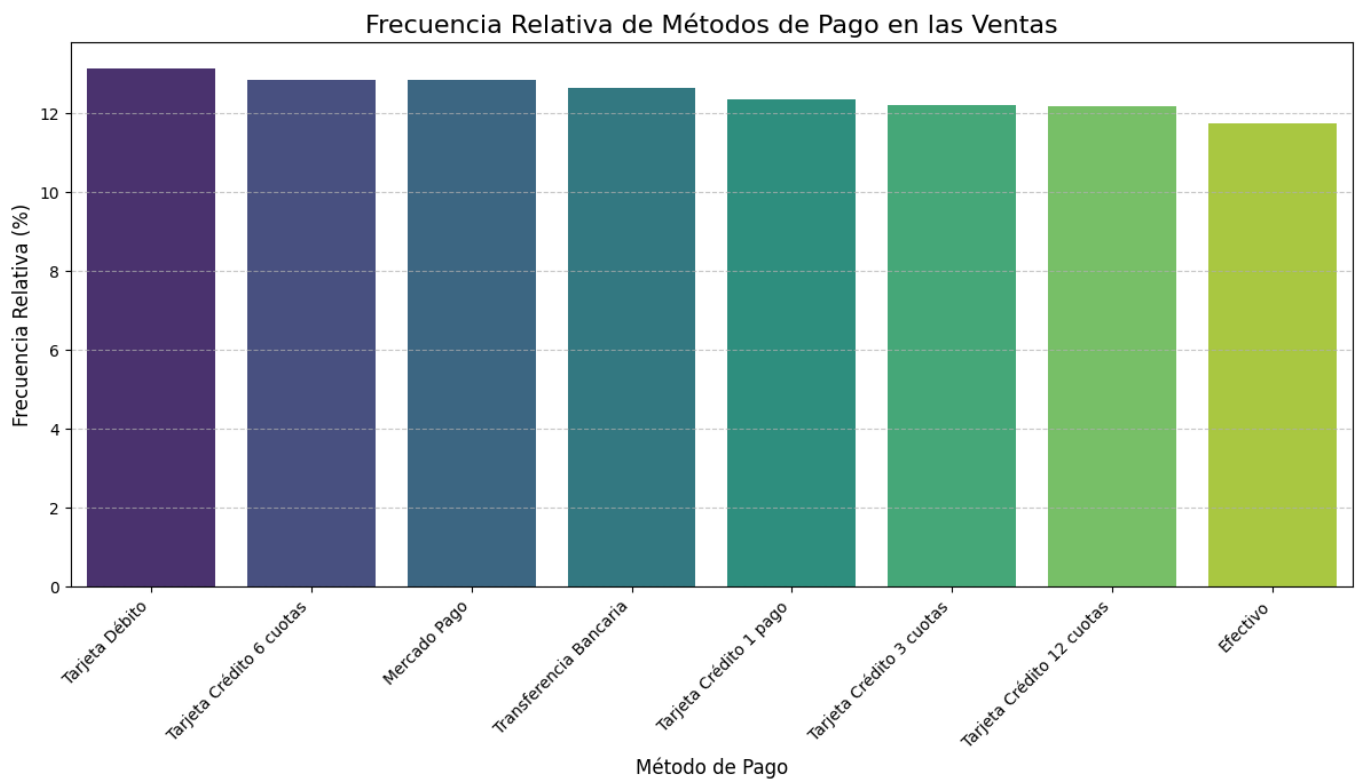
La peor reseña sobre el producto modelo P0005 dice: "Lamentablemente debo

decir que la calidad es muy mala con Licuadora. Es muy lento y difícil de limpiar. Se rompió a los pocos días de uso. Espero les sirva!"

El Agente comprendió lo que le estaba preguntando correctamente entendiendo el historial y el contexto de la conversación.

Prueba de *Analytics_tool* para generar gráficos

```
consultar_agente("Gráfico de la frecuencia relativa de cada medio de pago en las ventas realizadas", steps=True)
```



Paso 1

Pensamiento: Action: analytics_tool
Action Input: Graficar la frecuencia relativa de cada medio de pago en las ventas realizadas.

Herramienta: analytics_tool

Input: Graficar la frecuencia relativa de cada medio de pago en las ventas realizadas.

Resultado: Gráfico generado correctamente.
generado mediante
import pandas as pd
import matplotlib.pyplot as plt

```
import seaborn as sns

resultado = ""

try:
    df_ventas = dfs['ventas']

    # Calcular la frecuencia relativa de cada método de pago
    frecuencia_relativa =
df_ventas['metodo_pago'].value_counts(normalize=True) * 100

    # Crear el gráfico de barras
    plt.figure(figsize=(12, 7))
    sns.barplot(x=frecuencia_relativa.index, y=frecuencia_relativa.values,
palette='viridis')

    # Añadir título y etiquetas
    plt.title('Frecuencia Relativa de Métodos de Pago en las Ventas',
fontsize=16)
    plt.xlabel('Método de Pago', fontsize=12)
    plt.ylabel('Frecuencia Relativa (%)', fontsize=12)
    plt.xticks(rotation=45, ha='right') # Rotar etiquetas para mejor
legibilidad
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.tight_layout() # Ajustar el diseño para evitar solapamientos

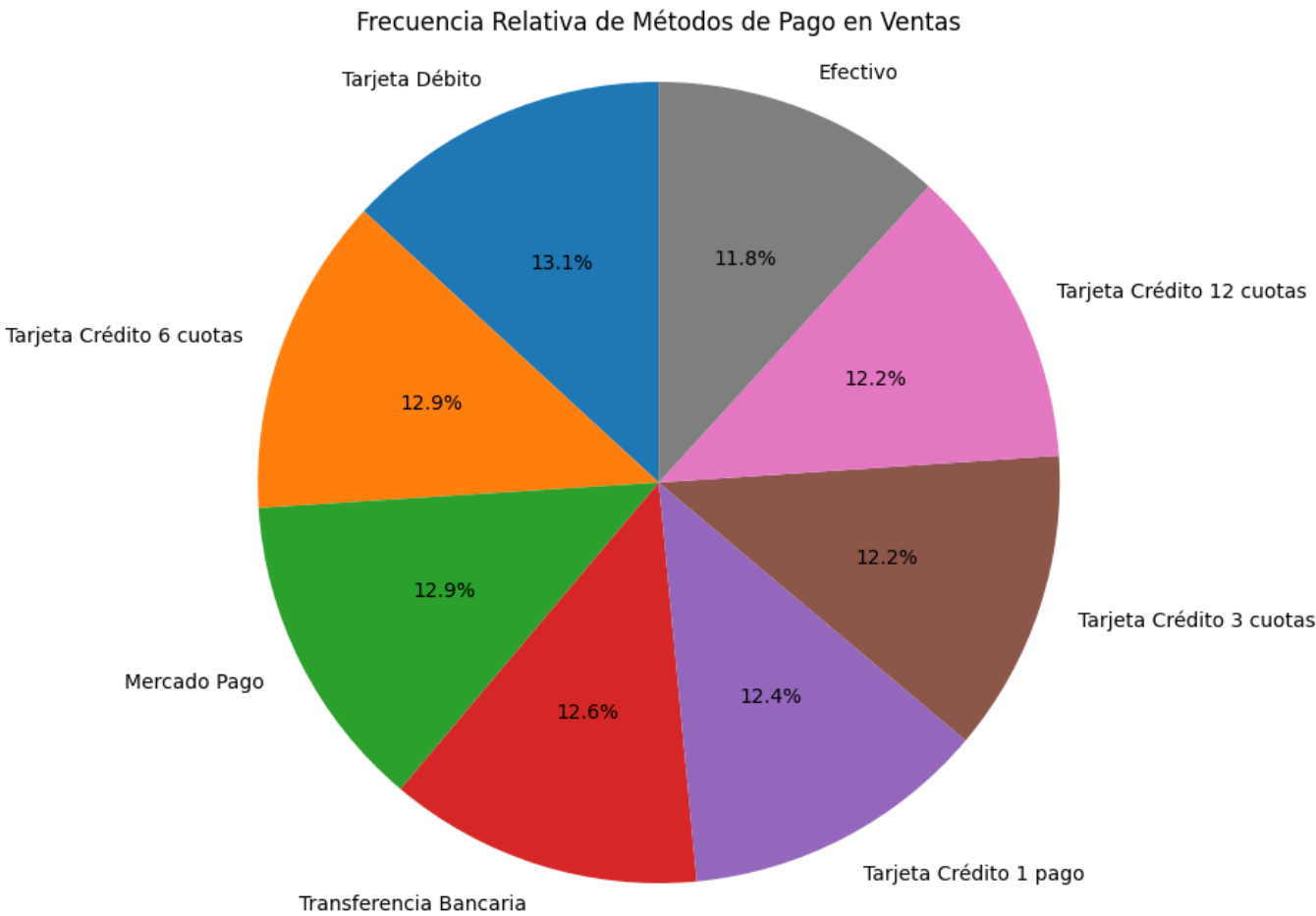
    # Mostrar el gráfico
    plt.show()
    resultado = "Gráfico generado correctamente."
except KeyError:
    resultado = "Error: La tabla 'ventas' o la columna 'metodo_pago' no se
encontraron en el diccionario 'dfs'."
except Exception as e:
    resultado = f"Error al generar el gráfico: {e}"

-----

Se ha generado un gráfico de la frecuencia relativa de cada medio de pago
en las ventas realizadas.
```

Prueba de historial

```
consultar_agente("Dame lo como piechart", steps=True)
```



Paso 1

Pensamiento: Action: analytics_tool
Action Input: Graficar la frecuencia relativa de cada medio de pago en las ventas realizadas como un gráfico de pastel (piechart).

Herramienta: analytics_tool

Input: Graficar la frecuencia relativa de cada medio de pago en las ventas realizadas como un gráfico de pastel (piechart).

Resultado: El gráfico se generó correctamente.generatedo mediante
import pandas as pd
import matplotlib.pyplot as plt

```
try:
    df_ventas = dfs['ventas']

    # Contar la frecuencia de cada método de pago
    frecuencia_metodos_pago = df_ventas['metodo_pago'].value_counts()

    # Calcular la frecuencia relativa (porcentaje)
    frecuencia_relativa = frecuencia_metodos_pago /
frecuencia_metodos_pago.sum() * 100

    # Generar el gráfico de pastel
```



```
plt.figure(figsize=(10, 8))
plt.pie(frecuencia_relativa, labels=frecuencia_relativa.index,
autopct='%1.1f%%', startangle=90)
plt.title('Frecuencia Relativa de Métodos de Pago en Ventas')
plt.axis('equal') # Asegura que el pastel sea un círculo.
plt.show()
resultado = "El gráfico se generó correctamente."
except KeyError:
    resultado = "Falló: No se encontró la tabla 'ventas' en el diccionario 'dfs'."
except Exception as e:
    resultado = f"Falló: Ocurrió un error inesperado - {e}"
```

Se ha generado un gráfico de pastel de la frecuencia relativa de cada medio de pago en las ventas realizadas.

Los gráficos generados son correctos, y entendió correctamente que le estaba pidiendo otro tipo de gráfico sobre la misma información de antes.

Más pruebas

Ahora sin mostrar los pasos intermedios *steps=False* para analizar únicamente resultados limpios

```
consultar_agente("Cual es el ratio ventas/devoluciones de cada sucursal?")
```

A continuación se presenta el ratio ventas/devoluciones para cada sucursal:

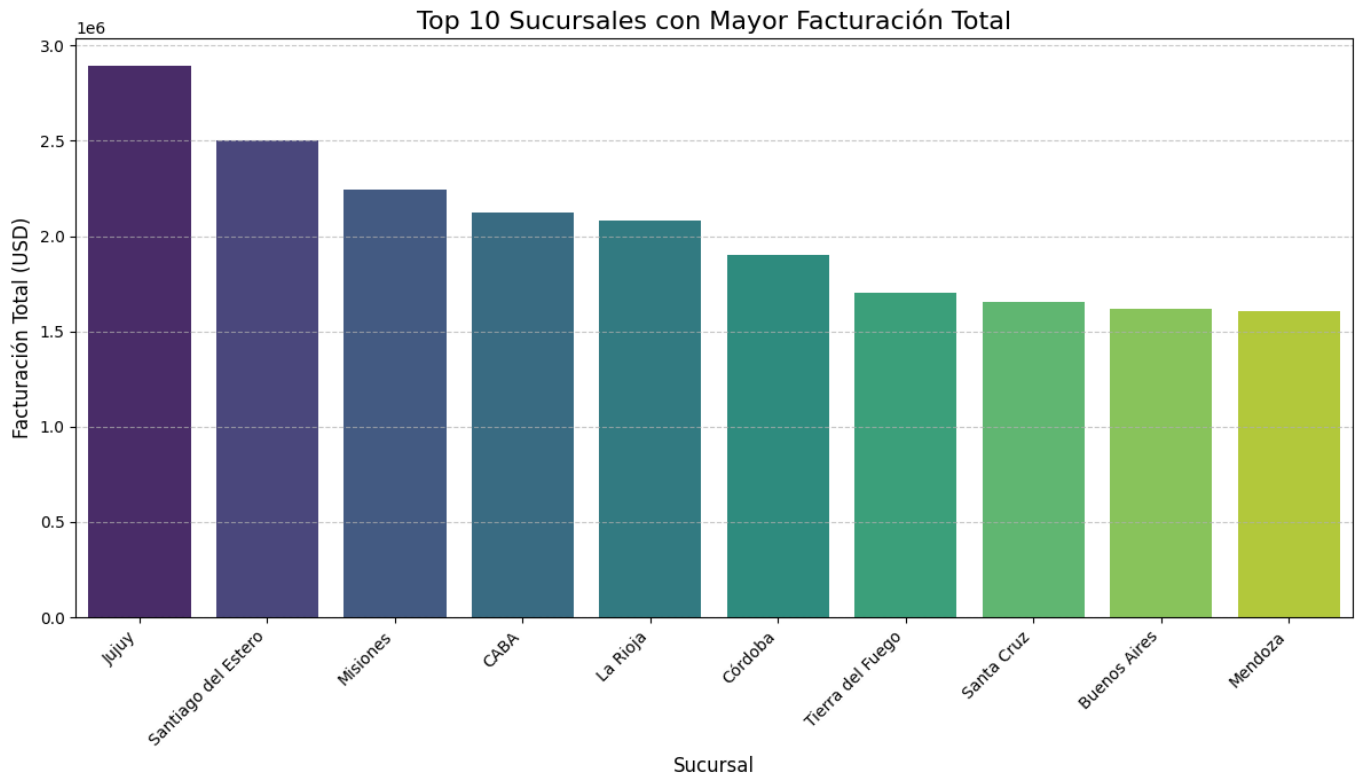
```
* **Buenos Aires:** 4.43
* **CABA:** 5.33
* **Catamarca:** 5.88
* **Chubut:** 5.86
* **Corrientes:** 4.63
* **Córdoba:** 3.68
* **Entre Ríos:** 4.86
* **Formosa:** 4.21
* **Jujuy:** 5.25
* **La Pampa:** 7.46
* **La Rioja:** 5.81
* **Mendoza:** 4.33
* **Misiones:** 5.09
* **Neuquén:** 4.15
* **Río Negro:** 3.52
* **Salta:** 5.98
* **San Juan:** 4.59
* **San Luis:** 4.75
* **Santa Cruz:** 4.70
```

```
* **Santa Fe:** 4.88
* **Santiago del Estero:** 6.32
* **Tierra del Fuego:** 6.27
* **Tucumán:** 4.09
```

```
consultar_agente("Soy Juan Flores, recomiendame un producto basandote en mis compras mas recientes")
```

Basado en tus compras recientes de electrodomésticos de cocina como la licuadora, el microondas, la waflera, la olla de cocción lenta y el frigobar, te recomiendo la **Profesional Parrilla Eléctrica 3000** de ChefMaster. Sería un excelente complemento para tu cocina y te permitiría ampliar tus opciones de preparación de alimentos.

```
consultar_agente("Quiero saber cual es la sucursal que más facturó en total, tambien necesito un gráfico de barras comparando las ventas totales de las principales 10 sucursales para ver la diferencia")
```



La sucursal que más facturó en total es Jujuy. Se ha generado un gráfico de barras comparando las ventas totales de las principales 10 sucursales.

```
consultar_agente("Busca un usuario que haya abierto un ticket de máxima
severidad. Dime su nombre, qué producto reclamó y busca si hay reseñas de
otros usuarios reportando ese mismo problema")
```

Un usuario que abrió un ticket de máxima severidad es Juan Castillo, quien reclamó por el producto Ventilador de Torre 2024. Se han encontrado reseñas de otros usuarios que reportan problemas similares, como que el producto "se rompió a los pocos días" y es "muy complicado y ruidoso".

```
consultar_agente("Que voltaje usa mi cafetera?")
```

La Deluxe Cafetera 2024 usa 12V. La Premium Cafetera también funciona con 12V.

En este caso hubiera sido mejor que nos pregunte que modelo exacto de cafetera tenemos. Pero es bueno que haya incluido 2 modelos con sus respectivos voltajes en la respuesta en lugar de simplemente adivinar.

```
consultar_agente("Cuantos meses de garantía tiene mi secarropa P0259?")
```

Tu secarropa P0259 tiene 36 meses de garantía.

```
consultar_agente("Quien es el vendedor con mayor cantidad de tickets
abiertos de máxima severidad asignados? Listalos")
```

La vendedora con la mayor cantidad de tickets abiertos de máxima severidad asignados es Laura Álvarez. Los tickets son los siguientes:

- * TKT000758: Problema - No enciende, Severidad - Alta, Estado - Abierto
- * TKT001122: Problema - Defecto de fábrica, Severidad - Alta, Estado - Abierto
- * TKT001348: Problema - No enciende, Severidad - Alta, Estado - Abierto

* TKT000026: Problema - Defecto de fábrica, Severidad - Alta, Estado - Abierto

Este fue el ejemplo que motivó la implementación de *Instrospection* para generar el esquema del gestor de Grafos.

Caso problemático

```
consultar_agente("Decime quienes son los usuarios que mas compraron, cuanto compraron y quien les vendió", steps=True)

Paso 1
Pensamiento: Action: graph_search
Action Input: usuarios que mas compraron, el monto total de sus compras y los vendedores involucrados.
Herramienta: graph_search
Input: usuarios que mas compraron, el monto total de sus compras y los vendedores involucrados.
Resultado: [{"Usuario": 'Ignacio Castillo', 'montoTotalCompras': 270033.36000000003, 'vendedoresInvolucrados': ['Laura Rodriguez', 'Carlos Rojas', 'Benjamin Suárez', 'Emilia Mendoza', 'Florencia Rios', 'Jazmin Medina', 'Santiago Flores', 'Tomás Romero', 'Pablo Cruz', 'Gonzalo Cabrera']}

Los 3 usuarios que más compraron son:
* **Ignacio Castillo**: Compró un total de $270033.36. Los vendedores involucrados fueron: Laura Rodriguez, Carlos Rojas, Benjamin Suárez, Emilia Mendoza, Florencia Rios, Jazmin Medina, Santiago Flores, Tomás Romero, Pablo Cruz, Gonzalo Cabrera.
* **Pablo Cruz**: Compró un total de $249139.44. Los vendedores involucrados fueron: Carlos Flores, Renata Garcia, Thiago Rodriguez, Benjamin Suárez, Bianca Suárez, Milagros Hernández, Santiago Flores, Maria Benit
* **Gonzalo Cabrera**: Compró un total de $242176.87. Los vendedores involucrados fueron: Emanuel Navarro, Gonzalo Rios, Adrián Reyes, Martina Castillo, Facundo Fernández, Juan Diaz, Benjamin Álvarez, Laura Álvarez

historial_agente.pop(-1) # Borro ultima interacción del historial

('Decime quienes son los usuarios que mas compraron, cuanto compraron y quien les vendió',
'Los 3 usuarios que más compraron son:\n' '**Ignacio Castillo**: Compró un total de $270033.36. Los vendedores involucrados fueron: Laura Rodriguez, Carlos Rojas, Benjamin Suárez, Emilia Mendoza, Florencia Rios, ...

consultar_agente("Decime quienes son los usuarios que mas compraron, cuanto compraron y quien les vendió", steps=True)

Paso 1
Pensamiento: Question: Decime quienes son los usuarios que mas compraron, cuanto compraron y quien les vendió
Thought: El usuario quiere saber quiénes son los usuarios que más compraron, cuánto compraron y quién les vendió. Esto implica buscar relaciones entre usuarios, ventas y vendedores, y también sumar las cantidades de
Action: graph_search
Action Input: Usuarios que mas compraron, el monto total de sus compras y los vendedores que les vendieron.
Herramienta: graph_search
Input: Usuarios que mas compraron, el monto total de sus compras y los vendedores que les vendieron.
Resultado: [{"Usuario": 'Ignacio Castillo', 'MontoTotalCompras': 22502.780000000002, 'VendedoresQueAtendieron': ['Laura', 'Carlos', 'Benjamin', 'Emilia', 'Florencia', 'Jazmin', 'Santiago', 'Tomás', 'Joaquín', 'Ignacio', 'María Gutiérrez', 'Ana Castillo', 'Pablo Cruz', 'Francisco Romero']}

Los usuarios que más compraron son:
* **Ignacio Castillo** con un monto total de compras de $22502.78, atendido por Laura, Carlos, Benjamin, Emilia, Florencia, Jazmin, Santiago, Tomás, Joaquín, Ignacio.
* **María Gutiérrez** con un monto total de compras de $21978.10, atendido por Emanuel, Lorenzo, Martina, Pablo, Florencia, Tomás, Benjamin, Santiago, Jazmin.
* **Ana Castillo** con un monto total de compras de $21178.38, atendido por Emanuel, Adrián, Thiago, Julieta, Pablo, Miranda, Lucía, Gabriel, Matías, Tomás, Joaquín.
* **Pablo Cruz** con un monto total de compras de $20761.62, atendido por Carlos, Renata, Thiago, Benjamin, Bianca, Milagros, Santiago, Maria, Martina.
* **Francisco Romero** con un monto total de compras de $20556.98, atendido por Julieta, Pablo, Florencia, Gabriel, Thiago, Adrián, Ignacio.
```

Los montos totales calculados son inconsistentes, distintas ejecuciones limpias (borrando el historial de interacciones) producen resultados diferentes.

Comentarios

En desempeño del agente es muy bueno, entiende el contexto e historial y utiliza las herramientas provistas correctamente. En especial la herramienta de generación de gráficos resulta muy útil y fiable (ya que garantiza utilizar pandas y no correr riesgo de números mal calculados).

Posibles mejoras

- Forzar al agente a usar `table_search()` cuando necesite calcular un número, para prevenir que haga cálculos sobre datos del gestor de grafos y se equivoque. [Caso Problematico](#)
- Implementar respuesta por tokens (efecto máquina de escribir) para reducir posible ansiedad del usuario.
- Memoria persistente guardando el historial de interacciones en una base de datos.

Conclusiones

La capacidad del Agente para orquestar herramientas y sobretodo para generar visualizaciones gráficas mediante código Python, transforma al sistema de un simple chatbot de consultas a una herramienta de Business Intelligence (BI) conversacional. Esto ofrece un valor añadido significativo para la toma de decisiones. Por otro lado el agente permite la optimización de interacciones combinando varios pedidos/preguntas en una única consulta, demostró ser completamente capaz de entender estos casos y de identificar las herramientas correctas.

Modelos utilizados

A continuación se lista cada modelos utilizado en el desarrollo del trabajo y la razón de su elección.

LLM: Gemini 2.5 Flash (cloud)

Elegido por su buen desempeño general, velocidad y accesibilidad mediante api. Lo utilizo para:

- Generar consultas para la fuente de datos tabular (pandas)
- Generar código matplotlib para graficar datos tabulares (pandas + matplotlib)
- Generar consultas para la fuente de datos de grafos (cypher)
- Clasificador de intención para el RAG. (Métricas perfectas)
- "Cerebro" del Agente ReAct. Interpreta la query del usuario, genera los prompts para cada herramienta y genera la respuesta final.

Embeddings: gemma-300m (local)

Elegido por su excelente desempeño con una baja cantidad de parametros (300 millones), lo que permite ejecutarse rápido en el entorno de colab (GPU T4) y por estar entrenado en el idioma español. Utilizado para generar embeddings de manuales, reseñas y preguntas frecuentes, permitiendo la búsqueda semántica.

Clasificador de intención: SVM + TF-IDF (local)

Elegido por la agilidad de implementación, velocidad de ejecución y fiabilidad. Si el clasificar mediante LLM via API fallá por cualquier motivo, se que el RAG no dejará de funcionar gracias a este modelo, que no depende de internet, ni limites de tokens, ni requiere gran poder de cómputo.

Retriever: BM25 (local)

Elegido por ser la opción estandar para búsqueda por palabras clave y al ser un algoritmo ligero, se ejecuta muy rapidamente en el entorno de Google Colab. Utilizado para la implementación de búsqueda híbrida sobre la fuente de datos vectorial (Chroma). Útil para buscar por modelos específicos de productos.

Reranker: mmarco-mMiniLMv2-L12-H384 (local)

Elegido por ser el mejor modelo disponible en huggingface soporte soporte de idioma español, además al ser un modelo chico corre bien en el entorno de colab (GPU T4). Utilizado para implementar la búsqueda híbrida sobre la fuente de datos vectorial (Chroma), sirve como un filtro de ruido final, asegurandose de devolverle al llm los fragmentos realmente relevantes. (Agente ReAct)